ICS-344: Information Security

Course Project: DVSA Vulnerability Discovery and Remediation

Vulnerability Title: Lesson #9 — Vulnerable Dependencies

Field	Details
Course / Section	ICS-344 — Term 252
Student ID	s202250640
AWS Region	us-east-1 (N. Virginia)
Date	April 2026

Part 1) Goal and Vulnerability Summary

This lesson demonstrates the Vulnerable Dependencies vulnerability in DVSA, which is classified as OWASP A06:2021 — Vulnerable and Outdated Components. The DVSA Lambda function includes two known vulnerable libraries in its deployment package: node-serialize version 0.0.4 and node-jose version 0.11.0. These outdated dependencies introduce critical and high severity vulnerabilities into the serverless backend. The security impact ranges from Remote Code Execution through unsafe deserialization (node-serialize) to cryptographic weaknesses and prototype pollution (node-jose dependency tree). The fix involves completely removing node-serialize, which has no safe version, and upgrading node-jose to version 2.2.0 or later.

Part 2) Why This Works / Root Cause

The vulnerability exists because the Lambda deployment package was built with pinned versions of third-party libraries that have since been publicly disclosed as insecure. node-serialize 0.0.4 is fundamentally unsafe by design — it deserializes JavaScript functions (IIFEs) embedded in JSON, meaning any attacker-controlled input passed to unserialize() can result in arbitrary code execution on the server. There is no patch available for this package; it must be removed entirely.

node-jose 0.11.0 carries its own vulnerabilities and also pulls in outdated transitive dependencies including lodash.pick (Prototype Pollution), node-forge (multiple cryptographic vulnerabilities), uuid (missing buffer bounds check), and shell-quote (Command Injection in older versions). These are resolved by upgrading to node-jose 2.2.0 which ships with a modernized, audited dependency tree.

Part 3) Environment and Setup

- DVSA deployed on AWS — Region: us-east-1 (N. Virginia)
- Test folder: ~/lesson9 in AWS CloudShell
- package.json declares: node-serialize@0.0.4 and node-jose@0.11.0
- Tools used: AWS CloudShell, npm, npm audit
- Student ID: s202250640

Part 4) Reproduction Steps

Step 1 — Navigate to the lesson9 folder and inspect package.json:

```
cd ~/lesson9
cat package.json
```

Screenshot 1 — package.json showing both vulnerable dependencies:

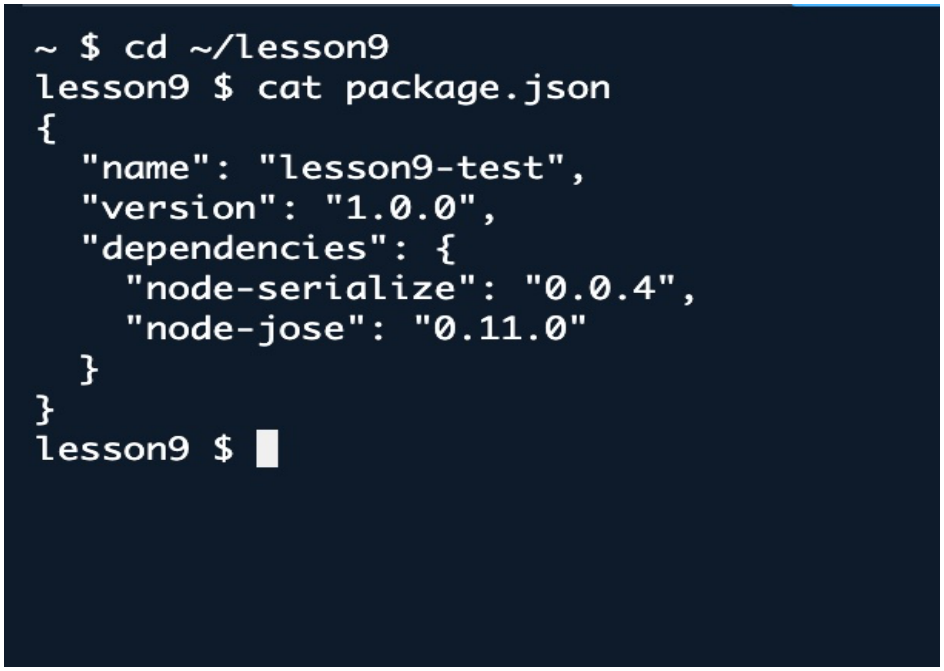

Figure 1: package.json — node-serialize@0.0.4 and node-jose@0.11.0 declared as dependencies

Step 2 — Run npm install to install the dependencies and observe vulnerability warnings:

```
npm install
```

Screenshot 2 — npm install output showing 6 vulnerabilities:

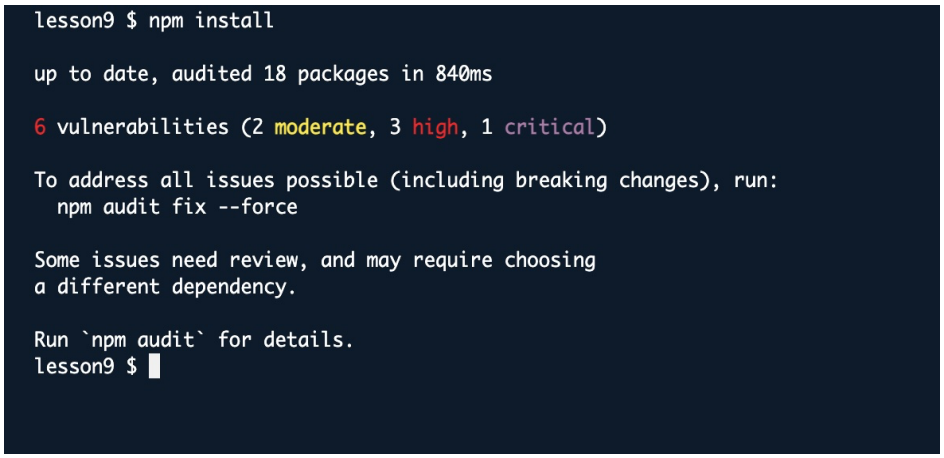

Figure 2: npm install — 6 vulnerabilities detected (2 moderate, 3 high, 1 critical)

Step 3 — Run npm audit to see the full vulnerability report:

```
npm audit
```

Screenshot 3a — npm audit report (first half) showing base64url, lodash.pick, and node-forge vulnerabilities:

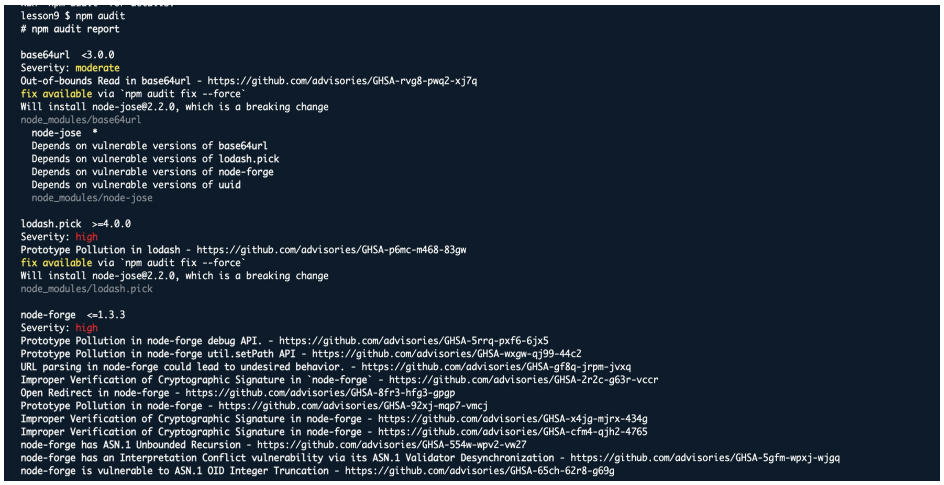

Figure 3: npm audit report (part 1) — moderate and high severity vulnerabilities in node-jose dependency tree

Screenshot 3b — npm audit report (second half) showing the critical node-serialize vulnerability:

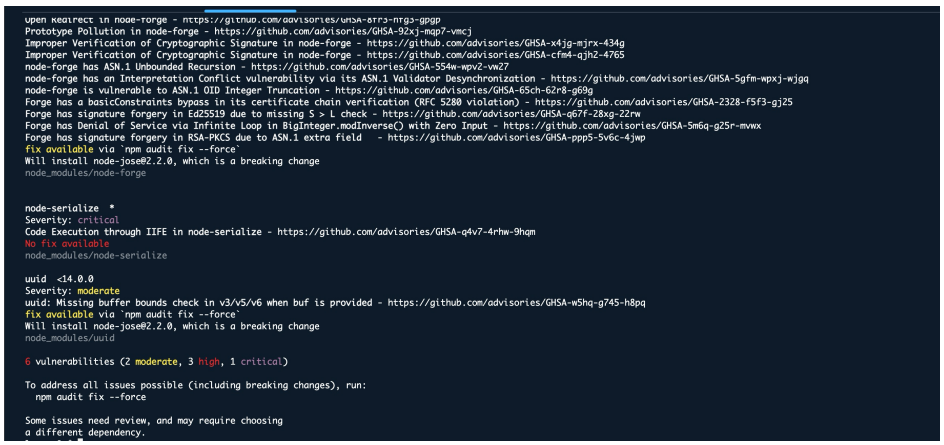

Figure 4: npm audit report (part 2) — node-serialize CRITICAL: Code Execution through IIFE, No fix available

Part 5) Evidence and Proof

The npm audit report confirms the following critical finding:

Key Finding: node-serialize — CRITICAL — Code Execution through IIFE — No fix available

node-serialize 0.0.4 allows an attacker to embed a JavaScript Immediately Invoked Function Expression (IIFE) inside a serialized JSON payload. When the backend calls unserialize() on attacker-controlled input, the embedded function executes immediately on the server. This is effectively Remote Code Execution (RCE). There is no patched version — the only remediation is complete removal of the package. This vulnerability is directly exploited in Lesson 1 (Code Injection) of the DVSA course.

Additional high severity findings from the node-jose 0.11.0 dependency tree:

- lodash.pick — Prototype Pollution (High) — attacker can inject properties into Object.prototype
- node-forge — Multiple cryptographic vulnerabilities (High) — improper signature verification, ASN.1 issues, denial of service

- base64url — Out-of-bounds Read (Moderate)
- uuid — Missing buffer bounds check (Moderate)

Part 6) Fix Strategy / Probable Mitigation

- Remove node-serialize completely — there is no safe version of this package. Any JSON deserialization needs should be replaced with JSON.parse() which does not execute functions.
- Upgrade node-jose to ^2.2.0 — the modern version ships with a clean, audited dependency tree that eliminates the high severity transitive vulnerabilities.
- Pin dependency versions in production — use exact versions or lock files (package-lock.json) to prevent accidental installation of vulnerable versions.
- Run npm audit in CI/CD pipelines — integrate dependency scanning into the deployment workflow so vulnerable packages are caught before reaching production Lambda.
- Apply the principle of least dependency — only include packages that are actively required. Unused or abandoned packages like node-serialize should be removed immediately.

Part 7) Fix Steps and Verification

Step 4 — Remove node-serialize completely:

```
npm uninstall node-serialize
```

Screenshot 4 — node-serialize removed, critical vulnerability eliminated:

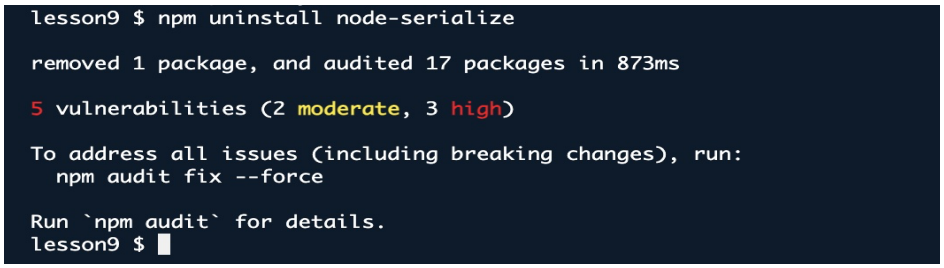

Figure 5: npm uninstall node-serialize — removed 1 package, vulnerabilities reduced to 5 (2 moderate, 3 high), critical is gone

Step 5 — Upgrade node-jose to version 2.2.0:

```
npm install node-jose@^2.2.0
```

Screenshot 5 — node-jose upgraded, high severity vulnerabilities resolved:

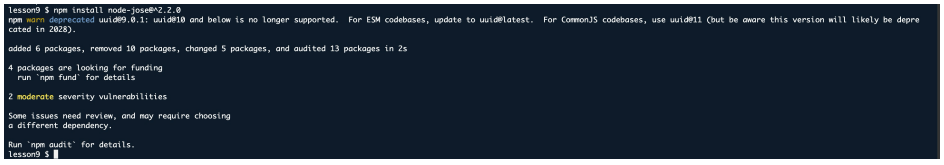

Figure 6: npm install node-jose@^2.2.0 — only 2 moderate severity vulnerabilities remain

Step 6 — Run npm audit after fix to confirm the remediation:

```
npm audit
```

Screenshot 6 — Final npm audit showing only 2 moderate vulnerabilities remain:

Figure 7: npm audit after fix — only 2 moderate (uuid) remain, no critical, no high

The post-fix audit confirms that the only remaining vulnerability is a moderate severity missing buffer bounds check in uuid, which is a transitive dependency of node-jose 2.2.0 with no fix currently available. The critical node-serialize vulnerability and all high severity node-jose dependency vulnerabilities have been fully resolved.

Part 8) Structured Operation and Security Analysis

Table A — Intended Logic vs Exploit Behavior

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Vulnerable Dependencies (A06:2021)	Only audited, up-to-date, and actively maintained packages should be included in Lambda deployment packages. No package with a known Critical or High CVE should be deployed to production.	npm audit report; DVSA package.json; node-serialize GitHub (abandoned, no patch); node-jose changelog; DVSA Lesson 1 exploit demonstrating node-serialize RCE.	A secure deployment would use JSON.parse() for deserialization and a modern version of node-jose. npm audit would return 0 vulnerabilities or only low severity findings.	npm audit confirms node-serialize 0.0.4 is Critical with No fix available. The library executes embedded JavaScript functions during deserialization, enabling RCE with attacker-controlled input to any API endpoint that calls unserialize().

Table B — Deviation Analysis and Fix

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Logging
Vulnerable Dependencies	The Lambda deployment package included node-serialize, an abandoned library with a Critical RCE vulnerability	Supply chain / dependency risk — outdated components	Removed node-serialize from package.json entirely. Upgraded node-jose from 0.11.0 to ^2.2.0 in package.json and ran npm	After fix: npm audit reports only 2 moderate vulnerabilities (uuid transitive dependency). Critical and all High severity findings are	Before: 6 vulnerabilities (2 moderate, 3 high, 1 critical). After: 2 moderate only. Critical node-serialize removed.

	and no patch, and node-jose 0.11.0 which carries High severity vulnerabilities through its dependency tree. Production code must never ship with known Critical vulnerabilities.		install to update the dependency tree.	eliminated. npm install no longer warns of critical issues.	High severity node-forge, lodash.pick, shell-quote fully resolved by node-jose upgrade.

Part 9) Takeaway / Lessons Learned

This lesson demonstrates that in serverless architectures, the Lambda deployment package is part of the attack surface. Every dependency bundled into a Lambda function is code that runs in your cloud environment, with access to the function's IAM role and any resources it can reach. A single abandoned library like node-serialize can give an attacker full Remote Code Execution inside your AWS infrastructure.

The key lesson is that dependency management is a security practice, not just a development convenience. Packages must be regularly audited using tools like npm audit. Abandoned packages with no patch available must be removed and replaced. Transitive dependencies — packages pulled in by your direct dependencies — are equally important and often overlooked.

The general secure design principles demonstrated here are: use the minimum necessary dependencies, prefer actively maintained libraries, integrate npm audit into CI/CD pipelines, and never deploy code with known Critical severity vulnerabilities. In cloud-native serverless applications where functions are event-driven and internet-facing, these practices are essential to preventing supply chain attacks.